BEGINNER'S ARCHITECTURE GUIDE

# Crossover Dynasty Technology Stack

How the website, Go API, database, live sports data, and a future Flutter mobile app fit together.

Prepared October 14, 2026 • Repository: `mosaicfantasy/dynasty`

# 1. The big picture

> **The central idea:** Crossover Dynasty has one backend that owns the data and fantasy rules. The website is one client of that backend. A Flutter application can become a second client without replacing the backend.




The website and mobile app should never connect directly to PostgreSQL. They ask the Go backend for information or request a change. The backend checks permissions and league rules before reading or changing data.

# 2. The current website: SvelteKit

The browser interface is built with:

- **Svelte 5** for user-interface components.
- **SvelteKit** for pages and browser routing.
- **TypeScript** for data types and client logic.
- **Vite** for the local development server and production build.

Pages live under `frontend/src/routes/`. A folder generally corresponds to a URL:

```
frontend/src/routes/
  players/       → /players
  league/        → /league
  draft/[id]/    → /draft/{id}
  player/[id]/   → /player/{id}
```

The frontend presents data and collects input. Important fantasy rules do not belong in the browser; they are enforced by the backend.

# 3. What an API is

An API is the agreed-upon language between a client and the server. Each request has an HTTP method, a path, and sometimes a JSON body.

```
GET    /api/players
GET    /api/standings
POST   /api/auth/login
PUT    /api/leagues/{id}/lineup
POST   /api/drafts/{id}/pick
```

The server normally answers with JSON. A player might look conceptually like this:

```
{
  "id": "player-id",
  "full_name": "Connor McDavid",
  "positions": ["C"]
}
```

### The frontend side of the API

`frontend/src/lib/api.ts` contains the website's TypeScript models and functions such as `getPlayers`, `getStandings`, `setLineup`, and `makePick`. This keeps individual pages from constructing network requests themselves.

### The backend side of the API

`backend/internal/web/server.go` maps methods and paths to Go handlers. Handlers in `backend/internal/web/` perform five small jobs:

1. Read the URL and JSON request.
2. Check authentication and permissions.
3. Validate the request's basic shape.
4. Call the appropriate domain service.
5. Return JSON or a readable error.

Handlers stay thin; most fantasy logic lives in domain packages.

# 4. The Go backend

The backend begins in `backend/main.go`. It connects the database, services, data providers, scheduled jobs, caching, websocket hub, and HTTP server.

| Package | Responsibility |
|---|---|
| `auth/` | Accounts, passwords and sessions |
| `dynasty/` | Dynasty, leagues and franchises |
| `roster/` | Adding, dropping and moving players |
| `lineup/` | Lineup rules and locking |
| `draft/` | Drafts, queues, picks and clocks |
| `trade/` | Trade validation and execution |
| `waiver/` | Waiver claims and processing |
| `scoring/` | Standings, matchups, playoffs and titles |
| `players/` | Player profiles, research and analytics |
| `ingest/` | External sports-data imports |
| `competition/` | Sport definitions and default rules |
| `web/` | HTTP API handlers and embedded website |

This division is important for mobile development. Flutter should not duplicate roster, scoring, waiver, or draft rules. It asks the server to perform an action, and the server decides whether that action is valid.

# 5. PostgreSQL, migrations and SQL queries

**PostgreSQL** is the permanent source of truth. It stores users, franchises, players, teams, rosters, lineups, drafts, trades, waivers, games, statistics, seasons, matchups, settings and transaction history.

Database structure changes are recorded as ordered SQL files under `backend/migrations/`. The backend applies outstanding migrations automatically when it starts.

### sqlc

Hand-written queries live under `backend/queries/`. The `sqlc` tool converts those queries into type-safe Go functions under `backend/internal/db/`.

**API handler → domain service → generated database function → PostgreSQL**

Generated `internal/db/*.sql.go` files should not normally be edited directly. Change the query and run `make gen`.

## 6. League rules and supported sports

Commissioner-configurable rules—including roster sizes, lineup slots, scoring, formats, waivers, drafts and trades—are defined and validated in `backend/internal/settings/settings.go`.

Sport-specific positions, statistics, default settings and data sources live in `backend/internal/competition/`. Most of the application works with general competition keys such as `nba`, `nhl` and `mlb`.

## 7. External sports data

Adapters under `backend/internal/ingest/` import real-world data from ESPN, MLB and NHL sources. They provide teams, players, prospects, injuries, schedules, results, game statistics and historical seasons.

Scheduled jobs run inside the Go server. Neither Svelte nor Flutter should call sports providers directly. This keeps provider-specific behavior and rate limiting in one place.

## 8. Live updates and websockets

A normal HTTP request returns once:

**Flutter → “Give me today's scores” → Go → JSON response**

A websocket remains open and allows the server to push changes:

**Flutter opens connection → Go sends current draft → picks and clock changes arrive live**

The project uses websockets for draft rooms, live scoreboards and individual games. Flutter can connect to these existing endpoints. While the app is in the background, mobile push notifications would eventually be needed for alerts such as “you are on the clock.”

## 9. Authentication and security

The website signs in with email and password. The server creates a signed JWT session and places it in a secure, HTTP-only cookie. Protected endpoints identify the manager through that session; commissioner endpoints add a permission check.

A Flutter app can initially use the same cookie. A stronger long-term mobile design would let the normal manager login return a bearer token, store it in the device's secure keychain, and attach it to API and websocket requests.

> **Never place these in the mobile app:** database credentials, the JWT signing key, the commissioner `ADMIN_TOKEN`, or private provider credentials. Those belong on the server.

## 10. Redis caching

Redis is an optional, disposable cache. It speeds up expensive reads—especially research results—but PostgreSQL remains authoritative. If Redis is cleared, league data is not lost. Clients never communicate with Redis directly.

## 11. Production deployment

**Internet → Caddy (HTTPS) → Go application → PostgreSQL + Redis**

Caddy provides HTTPS and forwards requests to Go. The Svelte site is compiled into static files and embedded inside the Go binary, so one application serves both the website and its API.

A Flutter app would be distributed separately through the Apple App Store and Google Play, then connect to the same deployment:

```
https://your-domain.com/api/players
https://your-domain.com/api/standings
wss://your-domain.com/api/drafts/{id}/ws
```

## 12. Where Flutter fits

Flutter would replace neither Go nor the website. It would add one shared Dart client that builds native iOS and Android applications.

| Flutter owns | Go backend owns |
|---|---|
| Screens, navigation and mobile interaction | Authentication and authorization |
| Local presentation state | Fantasy rules and validation |
| Calling HTTP and websocket endpoints | Database reads and writes |
| Securely retaining a manager session | Sports imports and scheduled jobs |
| Push-notification registration | Deciding when notifications should be sent |

The Dart client will need models corresponding to the API's JSON. Because the current contract lives mostly in a large TypeScript file, introducing an OpenAPI specification before two clients grow independently would reduce model drift.

## 13. Example: saving a lineup

1. A manager changes their lineup in Flutter.
2. Flutter sends `PUT /api/leagues/{id}/lineup` with JSON.
3. The Go handler confirms the manager is signed in.
4. The lineup service checks ownership, positions and locking rules.
5. A database transaction saves the valid lineup.
6. Go returns a success response.
7. Flutter updates the screen.

> **The boundary to remember:** Flutter is responsible for the mobile experience. Go remains responsible for truth, permissions, and fantasy rules. This keeps the website, iOS app and Android app consistent.

## 14. A sensible mobile starting scope

A first Flutter release does not need to reproduce every screen. A manageable first version could include:

1. Sign-in and session restoration.
2. My team, roster and player profiles.
3. Lineup management.
4. Standings, matchups and scores.
5. Live draft room.

Advanced research, custom charts and commissioner administration can remain on the website until the shared mobile foundation is stable.

## 15. Useful repository landmarks

| Path | What to find there |
|---|---|
| `README.md` | Setup, features and deployment overview |
| `backend/main.go` | Application wiring and background jobs |
| `backend/internal/web/server.go` | API route map |
| `backend/internal/settings/settings.go` | League settings and validation |
| `backend/internal/competition/` | Sport definitions and defaults |
| `backend/queries/` | Hand-written SQL queries |
| `backend/migrations/` | Database schema history |
| `frontend/src/lib/api.ts` | Current client types and API functions |
| `frontend/src/routes/` | Website pages |